2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)
September 24–28, 2017, Vancouver, BC, Canada

Sensor Fusion for Robot Control through Deep Reinforcement Learning

Steven Bohez, Tim Verbelen, Elias De Coninck, Bert Vankeirsbilck, Pieter Simoens and Bart Dhoedt [1]

Abstract— **Deep reinforcement learning is becoming increasingly popular for robot control algorithms, with the aim for a robot to self-learn useful feature representations from unstructured sensory input leading to the optimal actuation policy. In addition to sensors mounted on the robot, sensors might also be deployed in the environment, although these might need to be accessed via an unreliable wireless connection. In this paper, we demonstrate deep neural network architectures that are able to fuse information generated by multiple sensors and are robust to sensor failures at runtime. We evaluate our method on a search and pick task for a robot both in simulation and the real world.**

I. INTRODUCTION

Deep reinforcement learning has recently been applied to transform raw sensory input to control signals for various robotic tasks such as locomotion [1], grasping [2], manipulation [3], autonomous driving [4], etc. Generally the sensors and actuators are treated as a single, static system and the policy is trained on a fixed input size. With the current trend towards the Internet-of-Things, more and more sensor inputs can be provided by the environment in which the (mobile) robot operates. However, off-board sensors are accessed over an unreliable wireless link, and it might not be known upfront which sensors are actually operational.

In this paper, we propose a deep learning approach to fuse information coming from multiple, possibly remote sensors, while being robust to sensors being unavailable at runtime. A naive way of combining multiple inputs is by concatenating either the raw inputs or features calculated from these inputs in the neural network architecture [5], [6], [2], [7]. This approach however does not scale well in the number of inputs, and none define the behavior in the case of a missing input. Our contribution is twofold. First, we explore different neural network architectures for fusing multiple inputs that can be trained end-to-end with deep reinforcement learning methods. Second, we use a DropPath regularization method during training to make the neural network robust to missing inputs at runtime.

We test our approach on a search and pick scenario, in which a robot consisting of an omnidirectional platform with a 5-DOF manipulator has to drive towards a can and pick it up. As input, the robot receives lidar data from a sensor mounted on the robot, as well as sensors mounted in the environment. Our neural networks are trained and evaluated in simulation, and we also present preliminary results transferring the trained policies to a real Kuka YouBot.

The remainder of the paper is structured as follows. We discuss related work in Section II. In Section III we present the detailed setup of our experiments. Section IV presents our neural network architectures for fusing multiple inputs and DropPath regularization. In Section V we present our experimental results and finally Section VI concludes this paper.

[1]Authors are with Ghent University - imec, IDLab, Department of Information Technology. firstname.lastname@ugent.be

II. RELATED WORK

In this section, we will first discuss some important related work on both sensor fusion and deep reinforcement learning.

A. Sensor Fusion

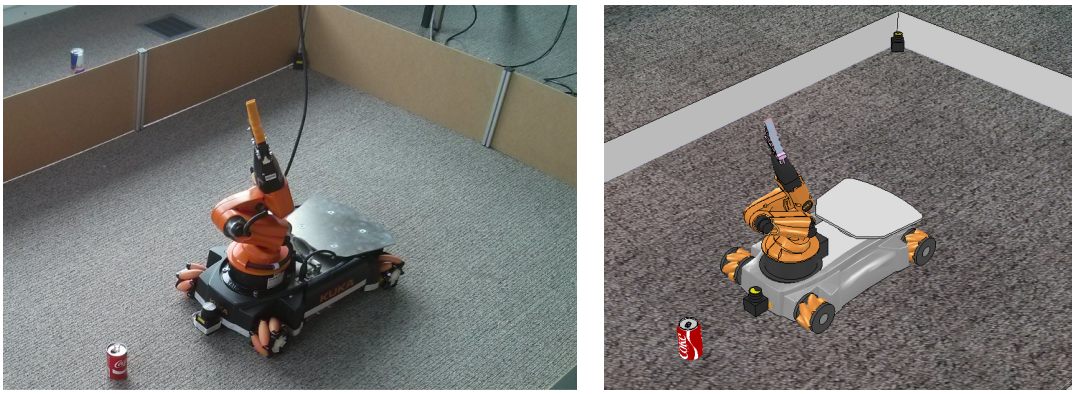
Combining information from several sources in order to get a unified picture has been a research topic for decades [8]. An often used sensor fusion technique is (Extended) Kalman Filtering and variants, which has been shown useful for use cases such as object position estimation [9], robot pose estimation [10], localization [11] and navigation [12]. In these cases the desired state representation based on the sensor input (e.g. the robot pose) is known and fixed upfront. However, in our approach we want to learn a policy end-to-end and the optimal feature representation to fuse the sensor input together is unknown upfront.

Work has been done on fusing multiple inputs using a deep learning approach, by learning a shared representation between all inputs [5]. Eitel et al. [6] present a two-stream Convolutional Neural Network (CNN) for RGB-D object recognition. Both RGB and D data are first separately processed by a pretrained CNN, whose outputs are then concatenated and converged in a fully connected layer with softmax classifier. In [2], RGB-D data is used for detecting robotic grasps. A structured regularization penalty is added when concatenating multi-modal features. Srivastava et al. [7] fuse bi-modal image-text and audio-video data using deep Boltzman machines. In this paper, we use neural networks to fuse lidar data from multiple sensors, which are trained end-to-end using a deep reinforcement learning approach.

B. Deep Reinforcement Learning

Deep reinforcement learning uses deep neural network models as function approximators. The neural network can either represent the policy itself by mapping observations to actions directly, or a value function from which a policy can be derived. In this paper we mainly consider the action-value function $Q(s,a)$, defined as

$$Q(s,a) = \mathbb{E}_\pi \left[\sum_{k=t}^{T} \gamma^{k-t} \cdot r_k \middle| s_t = s, a_t = a \right]. \quad (1)$$



(a) (b)

Fig. 1. Our real setup (a) with a Kuka YouBot in a rectangular arena, equipped with a Hokuyo lidar. Two additional Hokuyo sensors are deployed in the corners of the arena forming the diagonal. We replicated the same setup in the V-REP simulator (b) for training.

$Q(s,a)$ represents the expected discounted return, or Q-value, of executing an action a in a given state s and executing policy π afterwards. The reward r at each time step is discounted with a factor $\gamma \in [0,1]$, which gives more weight to short-term rewards. The policy $\pi(a|s)$ determines which action to take in each state. The optimal policy is found when following the optimal Q function $Q^*(s,a) = \max_\pi Q(s,a)$. Q^* satisfies the Bellman optimality equation

$$Q^*(s,a) = \mathbb{E}_{r,s'}\left[r + \gamma \cdot \max_{a'} Q^*\left(s',a'\right)\right], \tag{2}$$

with the expectation taken over the immediate reward r and next state s' when taking action a in state s. In Q-learning, Q^* is typically found by iteratively taking the action with the highest Q-value for each state, i.e. $\pi(a|s) = \arg\max_a Q(s,a)$, until convergence.

Deep Q-Networks (DQNs) extend Q-learning by approximating the Q-function by a deep neural network and have achieved state-of-the-art results in playing ATARI games [13]. This method uses the game pixel data as input and outputs discrete actions. The performance of this method was further improved using double Q-learning [14] and prioritized experience replay [15].

Recently, deep Q-learning was extended for continuous actions using Normalized Advantage Functions (NAFs) [3]. Other methods that work on continuous action spaces have an explicit policy function and directly optimize the expected reward using the policy gradient [1] or use an actor-critic approach in which a separate actor and critic neural network are used [16]. The actor network converts input to actions, while the critic network evaluates the actor network and provides gradient information. This approach can also be extended to stochastic policies [17].

In the remainder of this paper we will use the DQN algorithm for training our policies. However, our neural network architecture for fusing multiple inputs might equally well be applied to the other deep reinforcement learning algorithms.

III. ENVIRONMENT SETUP

As a test case for our sensor fusion approach, we have created a controlled environment for a search and pick task. Our setup is depicted in Figure 1(a). We created a 1.8 by 2.7 meter rectangular arena in which a Kuka YouBot is placed. The robot consists of an omni-directional base and a 5-DOF manipulator. A two-finger soft gripper is mounted on the manipulator to facilitate object picking. As a pick target we use soda cans. Sensor information is provided by Hokuyo URG-04LX-UG01 1D lidar sensors, of which one is mounted on the robot base, and two more are mounted on non-adjacent the arena corners. All sensors are configured to have a 180 degree field of view. An Nvidia Jetson TX1 embedded GPU is mounted inside the YouBot base and provides the compute power required for evaluating the policy. Each environment lidar is attached to a Raspberry Pi 2 that is connected over WiFi to the Jetson board. Both robot and sensors are controlled using the Robot Operating System (ROS) [18].

We have also replicated this environment in simulation as shown on Figure 1(b), using the V-REP simulator [19]. Within the simulator, the robot and sensors expose exactly the same interface via ROS. This allows us to control the simulated environment in the same way as the real world.

IV. NEURAL NETWORK ARCHITECTURES FOR DEEP RL SENSOR FUSION

We will use a DQN [13] to approximate and maximize the action-value function Q and derive the policy.

In our case the state s is determined by the observations of the three lidar sensors. Hence, our DQN will consist of three inputs, one for each sensor, and of one output: the Q values for each of the actions. For simplicity we have discretized the action space to seven actions for the robot: move left/right, move forward/backward, rotate clockwise/counterclockwise and execute a (hard-coded) grip. Not only do we want to fuse the input coming from the three sensors, we also want to be robust against sensor failures, since two of the sensors are not wired to the robot. However, we want to avoid unnecessarily increasing the number of weights when new sensors are added.

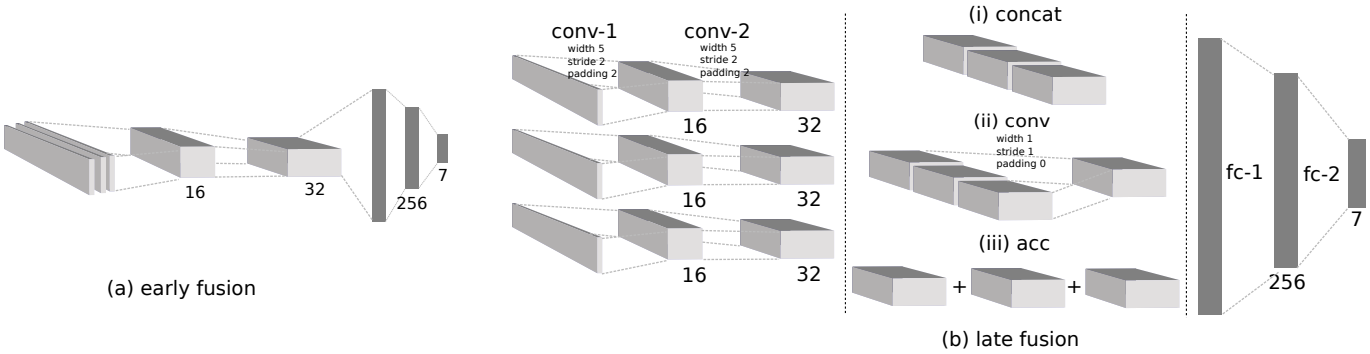


Fig. 2. The early fusion architectures (a) concatenate all inputs as feature planes. The late fusion architectures (b) start with two convolution layers for each sensor input, whose activations are then merged before the fully connected layers. We present three types of late fusion: (i) simply concatenating the convolution outputs, (ii) reducing the concatenated feature planes with a 1x1 convolution, and (iii) accumulating the convolution outputs.

There are multiple ways to combine multiple inputs in a neural network. As also pointed out by [2], inputs can be concatenated to a single input for a neural network. This approach will be denoted as "early fusion". A second approach is to first process the inputs in separate neural network stacks that are later combined into a number of common layers, which we will call "late fusion".

A. Early fusion

We start by defining a baseline model, that uses only the sensor mounted on the robot as input. Similar to [13], this model starts with two convolution layers, the first one with 16 filters, the second one with 32 filters, followed by a fully connected layer with 256 hidden units and a final fully connected layer that produces 7 outputs. In each layer we use ReLU activations.

Next, we add the two additional sensor inputs by early fusing. A first, small model is similar to single-sensor model, but now the additional sensor inputs are concatenated as additional feature planes before the first convolution. A second, large model triples the number of filters and hidden units in the fully connected layer, increasing the capacity of the network to compensate for the additional inputs.

B. Late fusion

In our late fusion models we replicate the first two convolutional layers for each sensor input, and fuse the generated feature representations before forwarding the result to the fully connected layers, as shown in Figure 2. In this case, the ReLU activation of the second convolution layer is applied after the merge, before the fully connected layer. A naive approach of fusing is to concatenate the convolution outputs into a single feature vector. This has the main disadvantage of dramatically increasing the number of weights of the next fully-connected layer as the number of sensors increases. This is because we fuse at the layer with the highest number of activations. In order to overcome this effect somewhat, we introduce as second model that adds an additional convolutional layer with width 1 to first collapse the concatenated feature planes. Finally, we propose a model that accumulates the convolution outputs directly, which requires no additional parameters but also limits the connectivity between the feature maps from different sensors. The intuition is that by training this model end-to-end, the output of the second convolution layer of each sensor will be a suitable state representation where each sensor can add weight to each of the features. Features that get enough weight from one or more sensor inputs will be activated for the fully connected layer.

An overview of all models and the number of parameters is listed in Table I.

C. DQN training and DropPath regularization

In DQN, we wish to learn the optimal Q-function Q^* using a neural network with parameters θ, $Q(s,a;\theta^*)$. This can be posed as a regression problem, where we minimize the following loss at iteration i:

$$L(\theta_i) = \mathbb{E}_{(s,a,r,s')\sim\mathscr{P}}\left[r+\gamma\cdot\max_{a'} Q\left(s',a';\bar{\theta}_i\right) - Q(s,a;\theta_i)\right]^2. \tag{3}$$

To train the network, we let the agent interact with the environment and store each transition tuple (s,a,r,s') in an experience pool or replay buffer $\mathscr{P}$. The Q-network is then updated by sampling a mini-batch from this experience pool. While θ_i are the current parameters of the Q-network, $\bar{\theta}_i$ are the parameters of the slow-moving target as in [13]:

$$\bar{\theta}_i = \theta_{k\cdot\lfloor i/k\rfloor}, \tag{4}$$

TABLE I
OVERVIEW OF THE DIFFERENT POLICY ARCHITECTURES.

Model	Architecture	#Params
Single	conv(16)>conv(32)>fc(256)>fc(7)	266887
Early (small)	conv(16)>conv(32)>fc(256)>fc(7)	267047
Early (large)	conv(48)>conv(96)>fc(256)>fc(7)	812391
Late (concat)	concat after 3x conv(32)	796551
Late (conv)	conv (width 1) (32) after concat	275367
Late (acc)	accumulate after 3x conv(32)	272263
Acc + DP	add DropPath before accumulate	272263

with k the update interval or the slow-moving target. Using an experience pool and slow-moving targets greatly increases the stability of learning a deep Q-network. Note that no gradient updates with respect to $\bar{\theta}_i$ are applied.

In our environment, chances are that one or more sensor inputs are unavailable at a given moment in time. In order to obtain a policy that is robust against unavailable sensor information, we apply DropPath regularization, similar to what is used in training FractalNets [20]. We apply DropPath in our *Late (acc)* model, by dropping each of the environment sensors input with a certain probability before the accumulation.

However, applying DropPath during the policy training with Equation 3 did not yield good results. The noise caused by the regularization adds additional instability to the regression problem. Instead, we first train a Q-network until convergence using Equation 3, resulting in trained parameters θ^*. Next, we add DropPath, and further train the Q-network by minimizing the loss

$$L(\theta_j) = \mathbb{E}_{(s,a,r,s')\sim\mathscr{D}}\left[Q(s,a;\theta^*) - Q_{DP}(s,a;\theta_j)\right]^2 \qquad (5)$$

where $Q(s,a;\theta^*)$ is the output of the original converged Q-network with all sensor information available and $Q_{DP}(s,a;\theta)$ is a Q-network with DropPath regularization. So intuitively, we first approximate the optimal Q-function using DQN, and next we try to reconstruct the obtained Q-function with missing sensor input.

V. EXPERIMENTAL RESULTS

To speed up the learning process, we simulate multiple agents that are uploading concurrently to a single, shared experience pool, and limit the resolution of the simulated lidar sensors to 128 scan points. The simulator steps at 100ms, which is the update rate of the lidar sensors, and each simulation step is recorded as experience pool sample. We run simulations on a compute cluster of 40 virtual machines with 2 CPUs and 4GB RAM each. We don't require GPU infrastructure for training since the limiting factor is the experience generation using the V-REP simulator.

For each simulation, a random start position and orientation for both the robot and soda can are generated. Note that the environment sensors are always in the same location. An optimal target location for the soda can is right in front of the robot, where a hard-coded grip action is able to pick up the can. The goal then becomes for the robot to position itself so that the can is on the optimal target spot. Hence we don't simulate the actual grip action, speeding up the simulation.

As the reward function, we use the normalized negative distance of the can position to the optimal target location of the robot. This results in a reward of -1 when the robot is located at one end of the arena and the can is at the opposite end, and a reward of 0 when the robot can actually fetch the can with the hard coded grip action. Note that while this reward requires additional sensors to be estimated in the real environment, we only consider the zero-shot transfer case in this paper and will not fine-tune model parameters based on real trajectories. A fixed reward of -1 is given when the robot collides with either the border or the can. Every roll-out is capped at 100 simulation steps, which is enough for the robot to fetch the can from any start configuration.

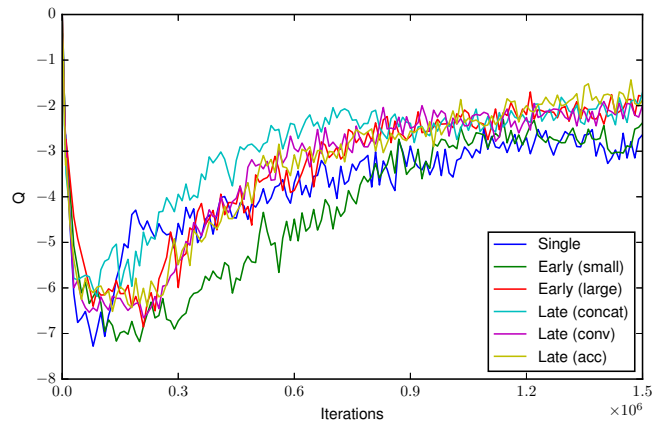


Fig. 3. Average $max_a Q(s,a)$ of mini-batches during training. After 1.5M iterations the Q-networks converge. Also note that the *Single* and *Early (small)* converge to a slightly lower Q-value.

All models are trained with DIANNE [21], a distributed deep learning framework built on Torch [22], that has built-in support for deep reinforcement learning. We update the neural network weights according to Equation 3, but we use double Q learning as described in [14], and pseudo-Huber loss instead of Mean Squared Error (MSE) as it is more resilient to outliers. For the gradient updates we use RMSProp with a learning rate of 0.0001. All agents use ε-greedy exploration with $\varepsilon = 1$ at the start and exponentially annealed to $\varepsilon = 0.1$. Our experience pool has a maximum size of 1 million samples, and is updated in a first-in-first-out manner. We train our models for 1.5 million batches of size 32. The average $max_a Q(s,a)$ per mini-batch is plotted on Figure 3. Note that the *Single* and *Early (small)* converge to a slightly lower Q-value, hinting at a lower success rate.

A. Early vs late fusion

To compare whether our models are able to fuse information coming from multiple sensors, we compare them against two baselines: one taking purely random actions, and the *Single* model that is trained on the robot's front-facing sensor only. To evaluate the models' effectiveness, we randomly generated 100 start poses for the robot and the can, evaluated each policy for each of these start configurations and capture the total return collected by the agent in each roll-out. Since the absolute reward depends on the start situation, we compare box plots showing the distribution of the rewards per roll-out.

Our results are shown in Figure 4. It is clear that the *Single* model performs much better than the pure random baseline. Adding information from the two additional lidar sensors introduces an additional leap in performance, showing that the models are indeed able to fuse the information. More specific, we see that in starting situations where the can is

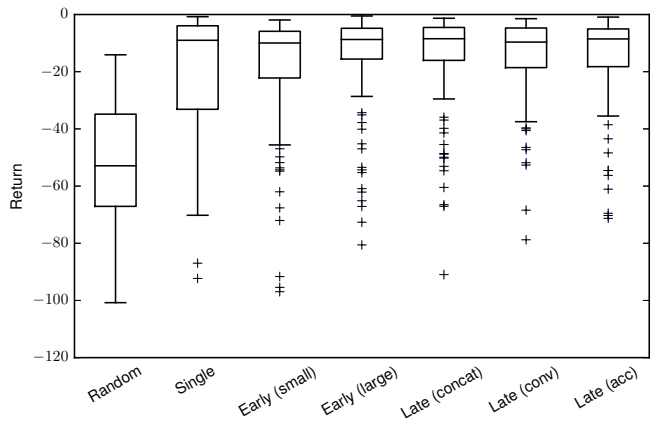


Fig. 4. Box plot of the returns achieved on 100 roll-outs for each of the models. The Q-network clearly benefits from additional sensor information, although there is little difference between late or early fusion. The models with more parameters perform slightly better.

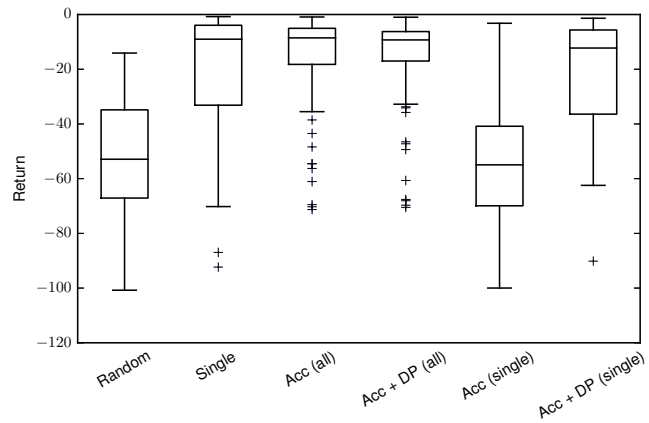


Fig. 5. When removing the remote sensor information from *Late (acc)*, this performs worse than random. However, when refining with DropPath regularization, we get similar results as training on single sensor data, while still performing better when all sensors are present.

positioned behind the YouBot (out of view of the front-facing sensor) the *Single* model is unable to correctly navigate towards the can, whereas this is more often the case in the models using fusing. We observed no significant difference in performance between early or late fusion models. The two models with a larger amount of parameters in the first linear layer (*Early (large)* and *Late (concat)*) perform slightly better than the others, most likely due to the increased model capacity. Also note the large negative tails of all box plots, indicating that there are certain start configurations that none of the models are able to solve correctly.

B. DropPath regularization

To evaluate DropPath regularization, we add DropPath to our *Late (acc)* model, with a drop rate of 0.5 for the remote sensors. Dropped sensors are replaced with zeros before the accumulate layer. The front facing sensor input is never dropped, as we assume this one is always available since it is directly wired into the robot. This rate balances equally between only having the front facing sensor, front facing + sensor 1, front facing + sensor 2 and all sensors available. We refine the *Late (acc)* model with additional training using Equation 5 for another 1M mini-batches on 5M experience pool samples generated with *Late (acc)*, resulting in the *Acc + DP* model.

We compare the regularized *Acc + DP* model to *Late (acc)* when using all three or only the front facing sensor in Figure 5. Since *Late (acc)* was trained on input data from all three sensors, we see that dropping the environment sensors has serious impact on the performance. When dropping the environment sensors, *Late (acc)* performs even worse than random. After regularization with DropPath however, we see that *Acc + DP* performs on par with *Single* when only the front facing sensor is available. When all sensors are available on the other hand, there is only a negligible performance difference with the original *Late (acc)* model, showing that DropPath itself did not have a detrimental effect on performance. This shows that our *Acc + DP* model is indeed able to fuse information from multiple sensor sources, while still being robust to missing sensor information at runtime.

To evaluate the final position of the robot, we have plotted on Figure 6 the cumulative distribution of the final distance of the can to the "optimal" grip location where a reward of 0 is received. All methods end within a 10 cm radius of this location in more than half of the cases. The models that only have a single sensor input perform worse than the methods fusing multiple sensors, often missing out in the cases that the can is positioned behind the robot.

C. Real-world results

When transferring trained models to our real-world setup without any fine-tuning, we experienced a number of problems. First, with a fixed grip action it is difficult for the robot to actually pick up a can with the two-finger gripper, as the robot has to be positioned very accurately. Second, the lidars' rays don't reflect on all materials, and reflection also depends on the angle. Especially the YouBot's base seemed not to reflect the lidar beams well, resulting in missing values in the sensor inputs (which are set to 0).

We mitigated the first issue by adapting the grip action to take feedback from the last lidar scan to estimate the exact position of the can to the robot base. To make the YouBot more "visible" by the environment sensors, we covered the base with masking tape. However, imperfect scans with missing measurements still occurred. Therefore, we also added DropOut [23] on the sensor inputs during our DropPath refinement, dropping lidar rays from the input with a probability of 0.025. This made our *Acc + DP* model more robust against the real-world lidar data. Results of our real-world experiments can be seen in the supplementary video material.

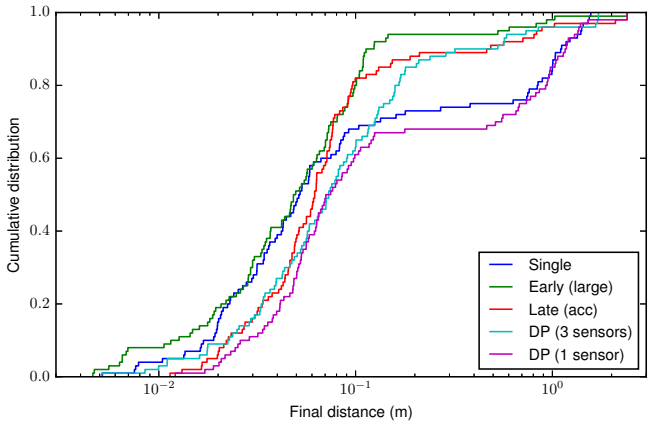


Fig. 6. Cumulative distribution of the final distance of the can to the optimal grip location.

VI. CONCLUSION

In this paper, we presented methods for fusing sensor input from multiple sources into a robot control policy using deep reinforcement learning. We evaluated different architectures on a search and pick task with a Kuka YouBot and multiple lidar sensors. We have shown that neural networks are indeed able to fuse information from multiple sources. We have found no significant performance difference between fusing early or late in the neural network. However, combining our late fusion approach by means of accumulation with DropPath regularization, we are able to train a policy network that leverages all sensor inputs, while still being robust when sensor input is missing.

As future work we will extend our evaluation to a wide variety of deep reinforcement learning algorithms, more specifically exploring recurrent policies that can better capture the environment dynamics, as well as using methods that work on continuous action spaces as this would be more suited for controlling the robot. Next, the environment should be extended to include more of the variations we see in the real setup, such as the data corruption we observe on the lidars, variations in sensor position and non-target objects within the workspace. Finally, we would also like to expand our approach to multi-modal sensor input, for example combining the lidar data with RGB(-D) camera input.

ACKNOWLEDGMENT

Steven Bohez is funded by Ph.D. grant of the Agency of Innovation by Science and Technology in Flanders (IWT). We gratefully acknowledge the support of NVIDIA Corporation with the donation of the Jetson TX1 used in this work.

REFERENCES

[1] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust Region Policy Optimization," in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015, pp. 1889–1897.

[2] I. Lenz, H. Lee, and A. Saxena, "Deep Learning for Detecting Robotic Grasps," *International Journal of Robotics Research*, vol. 34, no. 4-5, pp. 705–724, Apr. 2015. [Online]. Available: http://dx.doi.org/10.1177/0278364914549607

[3] S. Gu, E. Holly, T. P. Lillicrap, and S. Levine, "Deep Reinforcement Learning for Robotic Manipulation," *CoRR*, vol. abs/1610.00633, 2016. [Online]. Available: http://arxiv.org/abs/1610.00633

[4] R. Hadsell, A. Erkan, P. Sermanet, M. Scoffier, U. Muller, and Y. LeCun, "Deep belief net learning in a long-range vision system for autonomous off-road driving," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2008, pp. 628–633.

[5] J. Ngiam, A. Khosla, M. Kim, J. Nam, H. Lee, and A. Y. Ng, "Multimodal Deep Learning," in *Proceedings of the 28th International Conference on Machine Learning (ICML)*, 2011, pp. 689–696.

[6] A. Eitel, J. T. Springenberg, L. Spinello, M. Riedmiller, and W. Burgard, "Multimodal deep learning for robust RGB-D object recognition," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Sept 2015, pp. 681–687.

[7] N. Srivastava and R. Salakhutdinov, "Multimodal Learning with Deep Boltzmann Machines," *Journal of Machine Learning Research*, vol. 15, pp. 2949–2980, 2014. [Online]. Available: http://jmlr.org/papers/v15/srivastava14b.html

[8] B. Khaleghi, A. Khamis, F. O. Karray, and S. N. Razavi, "Multisensor data fusion: A review of the state-of-the-art," *Information Fusion*, vol. 14, no. 1, pp. 28 – 44, 2013.

[9] A. W. Stroupe, M. C. Martin, and T. Balch, "Distributed sensor fusion for object position estimation by multi-robot systems," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2001, pp. 1092–1098.

[10] Y. Dobrev, S. Flores, and M. Vossiek, "Multi-modal sensor fusion for indoor mobile robot pose estimation," in *IEEE/ION Position, Location and Navigation Symposium (PLANS)*, April 2016, pp. 553–556.

[11] V. Malyavej, W. Kumkeaw, and M. Aorpimai, "Indoor robot localization by RSSI/IMU sensor fusion," in *10th International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology*, May 2013, pp. 1–6.

[12] S. Lynen, M. W. Achtelik, S. Weiss, M. Chli, and R. Siegwart, "A robust and modular multi-sensor fusion approach applied to MAV navigation," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Nov 2013, pp. 3923–3929.

[13] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[14] H. V. Hasselt, "Double Q-learning," in *Advances in Neural Information Processing Systems 23*, 2010, pp. 2613–2621.

[15] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized Experience Replay," *CoRR*, vol. abs/1511.05952, 2015.

[16] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *International Conference on Learning Representations (ICLR)*, 2016.

[17] N. Heess, G. Wayne, D. Silver, T. Lillicrap, T. Erez, and Y. Tassa, "Learning Continuous Control Policies by Stochastic Value Gradients," in *Advances in Neural Information Processing Systems 28*, 2015, pp. 2944–2952.

[18] M. Quigley, K. Conley, B. P. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "ROS: an open-source Robot Operating System," in *ICRA Workshop on Open Source Software*, 2009.

[19] M. F. E. Rohmer, S. P. N. Singh, "V-REP: a Versatile and Scalable Robot Simulation Framework," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2013.

[20] G. Larsson, M. Maire, and G. Shakhnarovich, "FractalNet: Ultra-Deep Neural Networks without Residuals," *arXiv preprint arXiv:1605.07648*, 2016.

[21] E. De Coninck, T. Verbelen, B. Vankeirsbilck, S. Bohez, S. Leroux, and P. Simoens, "Dianne: Distributed artificial neural networks for the internet of things," in *Proceedings of the 2Nd Workshop on Middleware for Context-Aware Applications in the IoT*, ser. M4IoT 2015, 2015, pp. 19–24. [Online]. Available: http://doi.acm.org/10.1145/2836127.2836130

[22] R. Collobert, K. Kavukcuoglu, and C. Farabet, "Torch7: A matlab-like environment for machine learning," in *BigLearn, NIPS Workshop*, 2011.

[23] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014. [Online]. Available: http://jmlr.org/papers/v15/srivastava14a.html